Bug Report: CodeTroubleshoot

Book Catalog (Node.js) and Handwriting Recognition (Python)

Report date: October 10, 2026 | Prepared by: Claude | Source: CodeTroubleshoot-main.zip | Deliverable: CodeTroubleshoot-fixed.zip

1. Executive Summary

Both applications in the repository were reviewed in full: every backend, frontend, test and configuration file. **28 defects** were found and fixed: 16 in the Book Catalog app (Node.js) and 12 in the Handwriting Recognition app (Python).

The most serious problems made core features unusable:

- Editing a book was rejected with a 400 error for realistic values, because price and year were passed in swapped positions.
- The stock `-` button increased stock instead of reducing it.
- The category filter showed every category except the one selected.
- Downloaded OCR text was always empty, because the result file was opened in write mode and wiped.
- OCR returned garbage on every sample image because of five stacked pipeline defects.

All fixes were verified by direct execution where the environment allowed (see Section 8). Regression tests were added for each backend fix.

Area	Defects	Critical/High	Medium	Low
Book Catalog backend	6	4	1	1
Book Catalog frontend	7	0	4	3
Handwriting backend	7	5	2	0
Handwriting frontend	3	0	1	2
Handwriting text output	1	0	0	1
Page layout (CSS)	4	0	2	2
Total	**28**	**9**	**10**	**9**

2. Scope and Environment

Applications reviewed

- `nodejs/`: Express 4 REST API on `node:sqlite`, with a vanilla JavaScript single-page frontend (Ledger).
- `python/`: FastAPI service using OpenCV and Tesseract OCR, with a Jinja2 page and vanilla JavaScript frontend (Scribe).

Review method

1. Static reading of all source, test, config and frontend files.
2. Tracing each user action end to end (UI, API route, controller, service, repository, database).
3. Running the original code to reproduce behavior where possible (for example OCR on the four sample images).
4. Applying fixes, then re-running the same checks.

Test environment

- Node.js v22.22.2, Python 3.12.3, Tesseract installed.
- Express, Jest, FastAPI and pytest were not installable (no network access), so the full suites could not be run. See Section 8.

3. Severity Definitions

Severity	Meaning
High	A core feature is broken, returns wrong data, or loses data.
Medium	A feature works in some cases or the user is misled, but there is a workaround.
Low	Cosmetic, robustness or edge-case issue with limited impact.

4. Defect Summary

ID	Component	Title	Severity
BC-01	Routes	Search endpoint is unreachable	Medium

ID	Component	Title	Severity
BC-02	Controller	Update passes price and year in the wrong order	High
BC-03	Stock service	Decrease adds stock instead of subtracting	High
BC-04	Repository	Category filter is inverted	High
BC-05	Stats repository	"Copies in stock" ignores books with 1 copy	High
BC-06	Repository	LIKE wildcards in search text are not escaped	Low
BC-07	Frontend JS	Search runs one keystroke behind	Medium
BC-08	Frontend JS	Out-of-order responses overwrite newer results	Medium
BC-09	Frontend JS	Form errors appear behind the modal	Medium
BC-10	Frontend JS	Non-JSON or network failures are unhandled	Low
BC-11	Frontend JS	Decrease button is enabled at zero stock	Low
BC-12	Frontend JS	Empty-state colspan is hard-coded	Low
BC-13	CSS	Table wrapper has a fixed 1100px width	Medium
BC-14	CSS	Toolbar cannot wrap (min-width 900px)	Medium
BC-15	CSS	Duplicate white-space rule block	Low
BC-16	Server	No startup message	Low
HW-01	File utils	Download truncates and returns empty text	High
HW-02	Pipeline	Arguments to prepare_for_ocr are swapped	High
HW-03	Preprocessing	Inverted threshold gives white text on black	High
HW-04	Preprocessing	Images are stretched to 800x600	High
HW-05	OCR config	Page segmentation mode reads a single line	High
HW-06	OCR	Tesseract failures are not handled	Medium
HW-07	Text formatting	Line breaks are flattened	Low
HW-08	Frontend JS	No error handling; button can stay disabled	Medium
HW-09	Frontend JS	Preview URLs leak and stale image stays visible	Low
HW-10	Frontend JS	Copy button fails silently	Low
HW-11	Tests	No coverage for the defects above	Medium
BC-17	Tests	No coverage for update, stock, filter, stats	Medium

5. Book Catalog (Node.js): Detailed Findings

BC-01: Search endpoint is unreachable

- **Severity:** Medium
- **File:** `nodejs/src/routes/books.js`
- **Symptom:** `GET /api/books/search?q=dune` returns 404 "Book not found".
- **Root cause:** `router.get("/:id")` was registered before `router.get("/search")`. Express matches routes in order, so `/search` was captured as an id, failed `parseId`, and returned 404.
- **Fix:** Register `/search` before `/:id`.
- **Verification:** Confirmed by route-order inspection. A regression test was added (requires Express to run).

BC-02: Update passes price and year in the wrong order

- **Severity:** High
- **File:** `nodejs/src/controllers/bookController.js`
- **Symptom:** Saving an edit fails with "Published year is required" or "Published year is out of range" for normal values such as price 14.99, year 1969.
- **Root cause:** `bookService.updateBook` expects `(id, title, author, isbn, category, publishedYear, price, quantity)`, but the controller passed `body.price` and then `body.published_year`. The price was validated as a year and the year as a price.
- **Fix:** Pass `body.published_year` before `body.price`.
- **Verification:** Executed the controller with a PUT body. All fields now persist in the correct columns.

BC-03: Stock decrease adds instead of subtracting

- **Severity:** High
- **File:** `nodejs/src/services/stockService.js`
- **Symptom:** Clicking `-` increases the quantity. Stock can never be reduced, and the "Insufficient stock" error can never trigger.
- **Root cause:** `if (delta < 0) next = book.quantity + Math.abs(delta)` explicitly inverted the sign of negative adjustments.

- **Fix:** `const next = book.quantity + delta;`. The existing `next < 0` guard now works as intended.
- **Verification:** Decrease on quantity 4 gives 3. Decrease on quantity 0 returns 400.

BC-04: Category filter is inverted

- **Severity:** High
- **File:** `nodejs/src/repositories/bookRepository.js`
- **Symptom:** Choosing "Science" lists Fiction, History, Technology and Biography books, and hides Science.
- **Root cause:** The condition was `category != ?`.
- **Fix:** Changed to `category = ?`.
- **Verification:** Every result for category Science has category Science. Search text combined with a mismatched category returns no rows.

BC-05: "Copies in stock" ignores books with 1 copy

- **Severity:** High (wrong dashboard figure)
- **File:** `nodejs/src/repositories/statsRepository.js`
- **Symptom:** The dashboard total is lower than the real number of copies (for example, books with quantity 1 such as *Dune* contribute 0).
- **Root cause:** `SUM(CASE WHEN quantity > 1 THEN quantity ELSE 0 END)`.
- **Fix:** `COALESCE(SUM(quantity), 0)`.
- **Verification:** Dashboard total equals the sum of all book quantities.

BC-06: LIKE wildcards in search text are not escaped

- **Severity:** Low
- **File:** `nodejs/src/repositories/bookRepository.js`
- **Symptom:** Searching for `%` or `_` matches every book.
- **Root cause:** User text was interpolated into a LIKE pattern without escaping.
- **Fix:** Escape `\`, `%` and `_`, and add `ESCAPE '\'` to each LIKE clause.
- **Verification:** Searching `%` returns no rows. Searching `dune` still returns one row.

BC-07: Search runs one keystroke behind

- **Severity:** Medium
- **File:** `nodejs/frontend/js/app.js`

- **Symptom:** Typing "dune" searches for "dun". Clearing the box leaves the list filtered by the last character.
- **Root cause:** The handler read `pendingQuery` (the old value) before assigning the new one, then searched with the old value.
- **Fix:** Search with the current input value and remove `pendingQuery`. Input is now debounced by 250 ms.

BC-08: Out-of-order responses overwrite newer results

- **Severity:** Medium
- **File:** `nodejs/frontend/js/app.js`
- **Symptom:** The list occasionally shows results for an earlier query.
- **Root cause:** Every keystroke started a request, and nothing ensured the last response processed was the newest.
- **Fix:** A sequence counter (`loadSeq`) discards responses from superseded requests.

BC-09: Form errors appear behind the modal

- **Severity:** Medium
- **Files:** `nodejs/frontend/js/app.js` , `nodejs/frontend/index.html`
- **Symptom:** On a duplicate ISBN or validation error the dialog stays open and shows nothing.
- **Root cause:** `showError` writes to `#message` , which is on the page behind the modal backdrop.
- **Fix:** Added `<p id="form-message">` inside the dialog and route save errors to it. The script creates the element if it is missing.

BC-10: Non-JSON or network failures are unhandled

- **Severity:** Low
- **File:** `nodejs/frontend/js/app.js`
- **Symptom:** If the server is down or returns an HTML error page, nothing is shown and the console reports an unhandled rejection.
- **Root cause:** `response.json()` throws on non-JSON bodies, and `fetch` rejections were not caught.
- **Fix:** A `request()` helper wraps every call. `readJson` is exception-safe and returns a readable error such as "Could not reach the server".

BC-11: Decrease button is enabled at zero stock

- **Severity:** Low
- **File:** `nodejs/frontend/js/app.js`
- **Fix:** The `-` button is rendered disabled when quantity is 0 or less. The row click handler also ignores disabled buttons.

BC-12: Empty-state colspan is hard-coded

- **Severity:** Low
- **File:** `nodejs/frontend/js/app.js`
- **Fix:** Replaced the literal `7` with a `COLUMN_COUNT` constant.

BC-13 to BC-15: CSS layout defects

- **Severity:** Medium (BC-13, BC-14), Low (BC-15)
- **File:** `nodejs/frontend/css/styles.css`
- **Symptoms:** On screens narrower than about 1150px the page scrolls sideways, the table is cut off, and the toolbar cannot fit.
- **Root causes:**
  - `.table-wrap` had `width: 1100px; overflow: visible`.
  - `.toolbar` had `flex-wrap: nowrap; min-width: 900px`.
  - The `white-space: nowrap` rule was declared in a separate duplicate block.
- **Fixes:**
  - `.table-wrap` is now `width: 100%; overflow-x: auto`, and the table has `min-width: 720px`, so only the table scrolls.
  - `.toolbar` now uses `flex-wrap: wrap` with no minimum width.
  - The duplicate rule was merged. A style for `button:disabled` was added.

BC-16: No startup message

- **Severity:** Low
- **File:** `nodejs/src/server.js`
- **Fix:** The server logs `Listening on http://localhost:PORT` once it is ready.

BC-17: Missing test coverage

- **Severity:** Medium
- **File:** `nodejs/tests/books.test.js`

- **Gap:** There were no tests for update, stock decrease, category filtering, the search route or dashboard totals. These gaps are why BC-01 to BC-05 were not caught.
- **Fix:** Nine tests added (listed in Section 7).

6. Handwriting Recognition (Python): Detailed Findings

On the four bundled sample images the original pipeline produced `'Wun'`, `'cone'`, `'samp'` and `'vide an ing'`. After HW-01 to HW-05 were fixed, the output was `'The quick brown fox jumps'`, `'Hello from the workshop / Please review this note / Thank you'`, `'Tall / sample / page'` and `'Wide sample line'`. The individual contribution of each fix to accuracy was not measured separately.

HW-01: Download truncates and returns empty text

- **Severity:** High (data loss)
- **File:** `python/app/utils/files.py`
- **Symptom:** "Download text" returns an empty file, and the stored result is erased.
- **Root cause:** `read_result` opened the file with mode `"w+"`, which truncates it before reading.
- **Fix:** Open with mode `"r"`.
- **Verification:** A write followed by a read returns the original text. Invalid or unknown ids still raise `FileNotFoundError` (HTTP 404).

HW-02: Arguments to prepare_for_ocr are swapped

- **Severity:** High
- **File:** `python/app/services/pipeline.py`
- **Symptom:** OCR ran on the resized colour image, not the cleaned binary image that is shown to the user as "Processed image".
- **Root cause:** The function is defined as `prepare_for_ocr(original, preprocessed)` and returns its second argument, but was called as `(preprocessed, resized)`.
- **Fix:** Call `prepare_for_ocr(resized, preprocessed)`.

HW-03: Inverted threshold gives white text on black

- **Severity:** High
- **File:** `python/app/preprocessing/image_ops.py`
- **Symptom:** The processed image has white text on a black background, which Tesseract reads poorly.

- **Root cause:** `cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU`.
- **Fix:** `cv2.THRESH_BINARY + cv2.THRESH_OTSU`, which gives dark text on a light background.
- **Verification:** On a synthetic image the background pixel is 255 and the ink pixel is 0.

HW-04: Images are stretched to 800x600

- **Severity:** High
- **File:** `python/app/preprocessing/image_ops.py`
- **Symptom:** Wide or tall notes are distorted before OCR, which damages letter shapes.
- **Root cause:** `cv2.resize(image, (800, 600))` ignored the original aspect ratio.
- **Fix:** Scale by `min(max_width / width, max_height / height)`, using `INTER_AREA` when shrinking and `INTER_CUBIC` when enlarging.
- **Verification:** 480x1200 becomes 240x600, 1400x280 becomes 800x160, and 800x600 is unchanged.

HW-05: Page segmentation mode reads a single line

- **Severity:** High
- **File:** `python/app/recognition/ocr.py`
- **Symptom:** Multi-line notes return only one line, or garbage.
- **Root cause:** `--psm 7` tells Tesseract the image is a single text line.
- **Fix:** `--oem 3 --psm 6` (a single uniform block of text).

HW-06: Tesseract failures are not handled

- **Severity:** Medium
- **File:** `python/app/recognition/ocr.py`
- **Symptom:** If Tesseract is not installed or crashes, the API returns an unhandled server error.
- **Root cause:** `TesseractNotFoundError` is an `OSError` and `TesseractError` is separate. Neither was caught, and the route only handles `RuntimeError`.
- **Fix:** `extract_text` converts both into `RuntimeError`, which the route returns as a JSON 500 error.

HW-07: Line breaks are flattened

- **Severity:** Low (behavior decision)
- **File:** `python/app/utils/text.py`

- **Symptom:** A three-line note appears as one long line in a multi-line text box.
- **Fix:** Lines are joined with newlines. To restore the old single-line output, change `"\n".join` back to `" ".join`.

HW-08: No error handling; button can stay disabled

- **Severity:** Medium
- **File:** `python/frontend/static/js/app.js`
- **Symptom:** If the request fails or returns non-JSON, the Process button stays disabled and the user sees nothing.
- **Fix:** Wrapped the request in `try/catch/finally`, added a safe `readJson`, and always re-enable the button.

HW-09: Preview URLs leak and a stale image stays visible

- **Severity:** Low
- **File:** `python/frontend/static/js/app.js`
- **Fix:** Revoke the previous object URL when a new file is picked, and clear the old processed image.

HW-10: Copy button fails silently

- **Severity:** Low
- **File:** `python/frontend/static/js/app.js`
- **Symptom:** The Clipboard API is unavailable on non-HTTPS pages or when permission is denied, and nothing happens.
- **Fix:** On failure the text is selected and the user is told to press Ctrl+C.

HW-11: Missing test coverage

- **Severity:** Medium
- **File:** `python/tests/test_recognize.py`
- **Gap:** No test checked download content, resize behavior, binarization polarity or the Tesseract-failure path.
- **Fix:** Five tests added (listed in Section 7).

7. Regression Tests Added

Node.js (`nodejs/tests/books.test.js`)

- Updates a book with every field in the right place

- Returns 404 when updating a missing book
- Decreases stock by one
- Rejects a stock decrease below zero
- Filters books by category
- Combines search text and category
- Treats `%` in search text literally
- Search endpoint is reachable
- Dashboard counts every copy in stock

Python (`python/tests/test_recognize.py`)

- Download returns the recognized text
- Download of an unknown result returns 404
- Resize keeps the aspect ratio
- Preprocessing gives dark text on a light background
- Tesseract failure returns a 500 JSON error

8. Verification Status

Check	Result
Syntax check of all JS and Python files	Passed
Node services, repositories and controllers run directly against an in-memory database (update, stock, filter, search, stats, duplicate ISBN)	Passed
Python OCR pipeline on the four sample images	Passed (all four read correctly)
Python file utilities, resize and binarization checks	Passed
`npm test` (Jest and Supertest)	**Not run** (dependencies unavailable offline)
`pytest` (FastAPI TestClient)	**Not run** (dependencies unavailable offline)
Browser testing of the frontends	**Not run**

The new HTTP-level tests, the route-order fix (BC-01) and the frontend changes are therefore untested in a live environment. Run the commands below before merging.

```
cd nodejs && npm install && npm test
cd python && pip install -r requirements.txt && pytest
```

9. Files Changed

nodejs/

- `src/routes/books.js`
- `src/controllers/bookController.js`
- `src/services/stockService.js`
- `src/repositories/bookRepository.js`
- `src/repositories/statsRepository.js`
- `src/server.js`
- `frontend/js/app.js`
- `frontend/css/styles.css`
- `frontend/index.html`
- `tests/books.test.js`

python/

- `app/preprocessing/image_ops.py`
- `app/recognition/ocr.py`
- `app/services/pipeline.py`
- `app/utils/files.py`
- `app/utils/text.py`
- `frontend/static/js/app.js`
- `tests/test_recognize.py`

10. Observations Not Changed

These are not defects in the current behavior but are worth tracking.

- `python/scripts/generate_samples.py` hard-codes Windows font paths. On other systems it falls back to a small bitmap font and produces poor sample images.
- `python/app/routes/recognize.py` reads the whole upload into memory before checking the 5 MB limit. Rejecting oversize requests earlier would reduce memory use under abuse.
- OCR result files accumulate in the temp directory with no cleanup.

- Neither API has authentication or rate limiting. This is acceptable for a local demo but not for public hosting.
- The Node README requires Node.js 22, but `node:sqlite` is still experimental and prints a warning on some 22.x versions.

11. Recommendations

1. Run both test suites and fix any environment-specific failures before release.
2. Run the tests in CI on every change. Most defects here (BC-02 to BC-05, HW-01) would have been caught by basic tests.
3. Add a few end-to-end tests that exercise the real OCR on the bundled samples, skipped when Tesseract is not installed.
4. Decide on the intended output format for multi-line OCR text (HW-07) and document it.
5. Review the items in Section 10 before any public deployment.